UNIVERSAL ASYNCHRONOUS RECEIVER TRANSMITTER (UART PROTOCOL) IMPLEMENTATION USING VERILOG

PREPARED BY:

Dharmi Patel

Contents

1. ABSTRACT

Universal Asynchronous Receiver Transmitter (UART) is a widely used serial communication protocol that enables data exchange between digital systems without requiring a shared clock signal. This project presents the design and simulation of a UART Transmitter and Receiver using Verilog HDL.

The UART module was developed to perform serial data transmission and reception using baud-rate-controlled communication. The transmitter converts parallel data into a serial bit stream by adding start and stop bits, while the receiver detects incoming serial data and reconstructs the original data byte. The design also includes interrupt generation to indicate successful data reception.

Functional verification was carried out using a Verilog testbench. A loopback configuration was implemented by connecting the transmitter output directly to the receiver input, allowing automatic verification of transmitted and received data. Simulation results confirmed correct UART frame generation, data reception, and reliable operation of the designed module.

2. PROJECT OBJECTIVES

- To design and implement a UART communication module using Verilog HDL for asynchronous serial data transmission and reception.
- To develop a robust transmitter and receiver architecture capable of reliable data exchange.
- To implement baud-rate-controlled communication for accurate UART frame generation and reception.
- To verify the functional correctness of the UART design through simulation and waveform analysis.
- To validate end-to-end serial communication using a loopback-based testing approach.

3. INTRODUCTION

Serial communication plays a vital role in modern digital and embedded systems by enabling efficient data exchange between devices using a limited number of communication lines. Among the various serial communication protocols, the Universal Asynchronous Receiver Transmitter (UART) is one of the most widely used due to its simplicity, reliability, and ease of implementation.

UART is an asynchronous communication protocol that allows data transmission and reception between two devices without requiring a shared clock signal. Instead, both communicating devices operate at a predefined baud rate, ensuring synchronized data transfer. A typical UART data frame consists of a start bit, data bits, an optional parity bit, and one or more stop bits, enabling accurate serial communication.

This project focuses on the design and simulation of a UART module using Verilog HDL. The design incorporates both transmitter and receiver functionalities, baud-rate-based communication, and interrupt generation for data reception. Functional verification was carried out using a Verilog testbench, where a loopback configuration was employed to validate successful serial data transmission and reception. The project demonstrates the implementation of UART communication principles and provides practical experience in digital design, protocol implementation, and simulation-based verification.

4. UART PROTOCOL OVERVIEW

Universal Asynchronous Receiver Transmitter (UART) is a serial communication protocol used for data exchange between digital devices. It enables asynchronous communication, meaning that no separate clock signal is required between the transmitter and receiver. Instead, both devices operate at the same baud rate for proper synchronization.

UART communication uses two lines: Transmit (TX) and Receive (RX). Data is transmitted serially in the form of a frame consisting of a start bit, data bits, an optional parity bit, and stop bit(s). The simplicity, low hardware requirements, and ease of implementation make UART one of the most widely used communication protocols in embedded systems, microcontrollers, and FPGA-based designs.

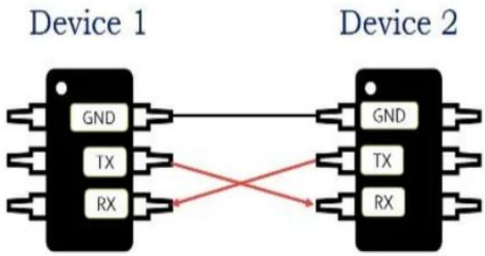

Figure 1 UART CONFIGURATION

4.1 UART Architecture

The UART architecture consists of a Transmitter, Receiver, Baud Rate Generator, and Control Logic. These blocks work together to perform asynchronous serial communication through the TXD and RXD lines.

The transmitter section stores parallel data in the Transmit Holding Register and serially shifts it through the TXD line using the Transmit Shift Register. The receiver section receives serial data through the RXD line, reconstructs the original data using the Receiver Shift Register, and stores it in the Receiver Holding Register.

The Baud Rate Generator produces timing signals required for data transmission and reception. It ensures that both the transmitter and receiver operate at the selected baud rate for reliable communication. Accurate baud-rate generation is essential for reliable data transmission and reception.

The Control Logic manages transmission, reception, data shifting, start and stop bit processing, and overall UART operation. Together, these blocks enable the conversion of parallel data into serial form for transmission and the reconstruction of serial data into parallel form during reception, allowing reliable asynchronous communication between digital systems.

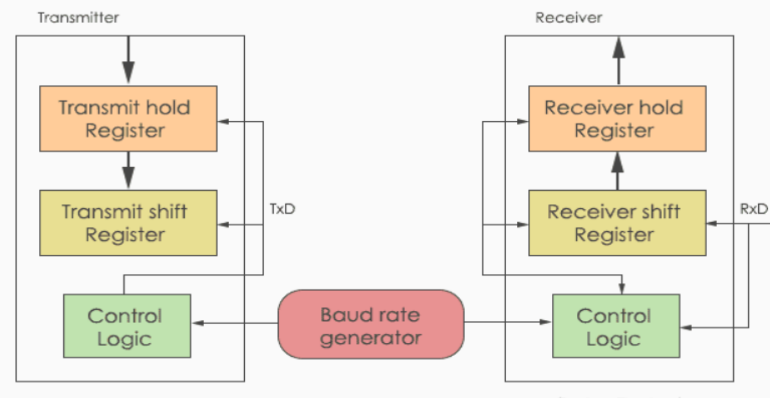

Figure 2 BLOCK DIAGRAM OF UART PROTOCOL

4.2 UART Communication Process

UART communication follows a predefined sequence for transmitting and receiving data. When no data is being transmitted, the communication line remains in the idle state at logic high. During idle state, the TX line stays at logic HIGH, and the receiver continuously monitors the line for the start of a new data frame.

Start Bit:
A UART transmission begins with a Start Bit. The transmitter drives the TX line LOW for one bit duration, indicating the start of a new data frame and allowing the receiver to synchronize with the incoming data.

Data Transmission
After the Start Bit, the transmitter sends the data bits serially. Typically, 8-bit data is transmitted, with the Least Significant Bit (LSB) sent first. The receiver samples each bit according to the configured baud rate and reconstructs the original data.

Stop Bit
Once all data bits have been transmitted, the transmitter sends a Stop Bit by driving the TX line HIGH. The Stop Bit marks the end of the data frame and allows the receiver to verify successful reception.

Data Reception
The receiver detects the Start Bit, receives the incoming serial bits, and stores the reconstructed data in the receive register. After successful Stop Bit detection, the received data becomes available for further processing.

After completion of the data frame, the communication line returns to the idle state and remains ready for the next transmission.

4.3 UART Frame Format

The UART frame implemented in this project consists of:

FIELD	SIZE
Start Bit	1 Bit
Data Bits	8 Bits
Stop Bit	1 Bit

Total Frame Length = 10 Bits

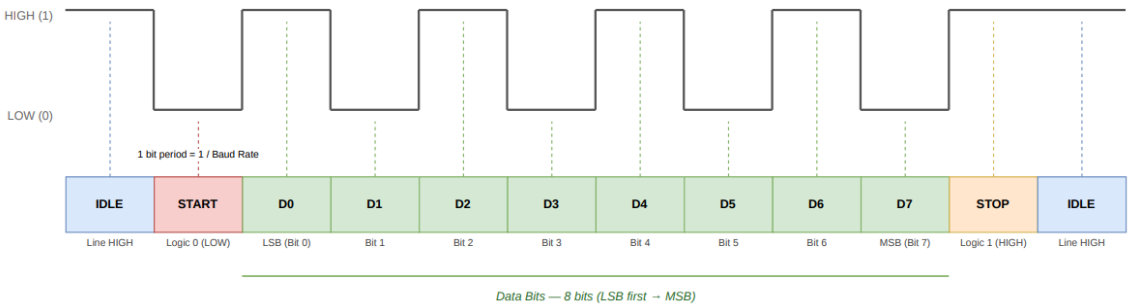

Figure 3 UART DATAFRAME

4.4 Baud Rate

Baud rate is the number of bits transmitted or received per second in a communication system. In UART communication, the baud rate determines the speed of data transfer between the transmitter and receiver. For reliable communication, both devices must operate at the same baud rate; otherwise, data may be received incorrectly.

In the proposed design, baud-rate timing is generated using an internal counter driven by the system clock. The counter divides the high-frequency clock to produce baud-rate ticks, which control the timing of data transmission and reception. These timing signals ensure that each bit is transmitted and sampled at the correct interval, enabling accurate and reliable UART communication.

Common UART baud rates include 9600, 19200, 38400, 57600, and 115200 bits per second (bps).

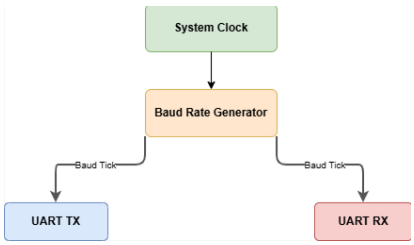

Figure 4 BAUDRATE GENERATION DIAGRAM

5. SYSTEM ARCHITECTURE

5.1 Block Diagram

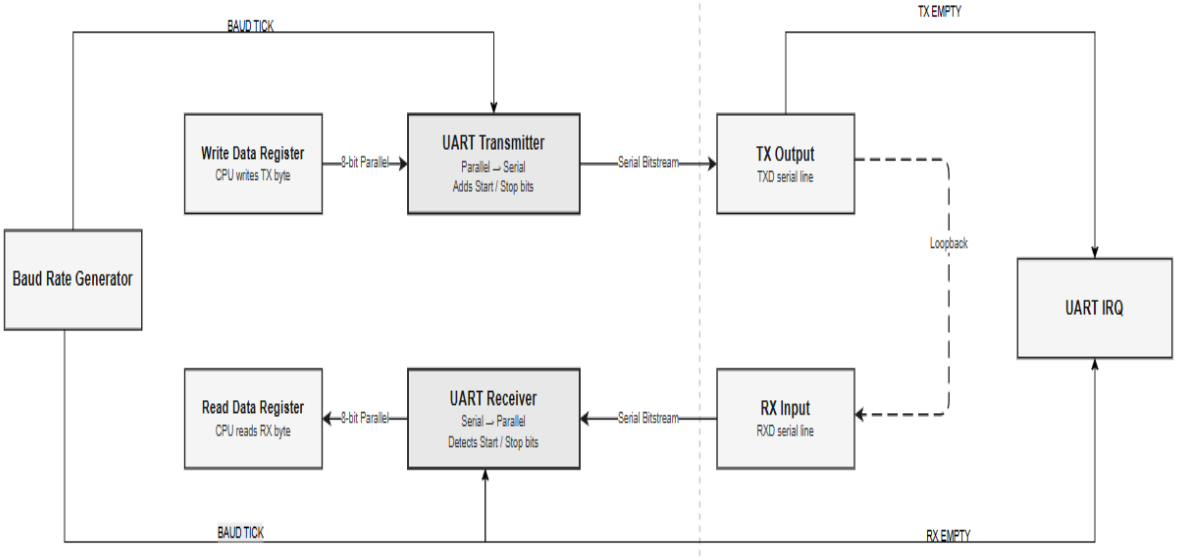

Figure 5 PROPOSED SYSTEM BLOCK DIAGRAM

5.2 Signal Interface Table

SIGNAL	DIRECTION	DESCRIPTION
clk	Input	System clock used for UART operation
reset	Input	Resets the UART module
address	Input	Selects UART register address
write_enable	Input	Enables write operation
write_data	Input	Data to be transmitted
read_data	Output	Data received by UART
tx	Output	Serial transmit line
rx	Input	Serial receive line
uart_irq	Output	Interrupt generated after data reception
baud_tick	Internal	Timing signal generated by baud rate counter

6. UART MODULE DESIGN

6.1 Register Map

The UART module uses a memory-mapped register interface to facilitate data transmission, data reception, status monitoring, and interrupt handling. Four registers are implemented in the design, each assigned a unique address.

REGISTER	ADDRESS	ACCESS	DESCRIPTION
REG_WDATA	0x00	Write	Transmit Data Register
REG_RDATA	0x04	Read	Receive Data Register
REG_READY	0x08	Read	Transmitter Status Register
REG_RXSTATUS	0x0C	Read	Receiver Interrupt Status Register

6.2 Bus Interface

The UART module uses a memory-mapped bus interface to communicate with the processor. This interface allows the processor to access UART registers for data transmission, data reception, and status monitoring through standard read and write operations.

The rw_address signal is used to select the desired register, while the write_request and read_request signals control write and read transactions, respectively. During a write operation, data is transferred from the processor to the selected UART register. During a read operation, the UART module places the requested register contents on the data bus. The write_response and read_response signals are used to indicate the successful completion of bus transactions.

The bus interface provides a simple and efficient mechanism for exchanging data and control information between the processor and the UART module.

6.3 Interrupt Generation

The UART module incorporates an interrupt mechanism to notify the processor when a valid data byte has been successfully received. After the receiver detects the Start Bit, receives all eight data bits, and verifies the Stop Bit, the received data is stored in the Receive Data Register.

Once the reception process is completed, the UART interrupt signal is asserted. This interrupt indicates that new data is available for processing and eliminates the need for continuous polling of the receiver status.

The interrupt remains active until it is acknowledged by the processor through the uart_irq_response signal or when the received data register is accessed. After acknowledgement, the interrupt is cleared and the receiver becomes ready to accept the next UART frame.

This interrupt-based approach improves communication efficiency and enables timely processing of received data.

7. UART TRANSMITTER DESIGN

7.1 Features

- Support for asynchronous serial communication.
- Transmission of 8-bit data frames.
- Automatic Start Bit and Stop Bit generation.
- Parallel-to-serial data conversion.
- Baud-rate-controlled transmission timing.
- Memory-mapped register-based data transmission.
- Transmission status monitoring through the Ready Register.
- Reliable serial data output through the UART TX line.

7.2 Working

The UART transmitter begins operation when the processor writes an 8-bit data value to the Transmit Data Register. Upon receiving a valid write request, the transmitter loads the data into the transmit register and constructs a UART frame consisting of one Start Bit, eight Data Bits, and one Stop Bit.

The frame is stored in a 10-bit transmit register. A baud counter generates the timing required for serial communication, while a bit counter tracks the number of bits remaining to be transmitted.

During each baud interval, the transmitter shifts one bit from the transmit register onto the UART TX line. The transmission starts with the Start Bit (logic LOW), followed by the eight data bits transmitted in LSB-first order, and finally the Stop Bit (logic HIGH).

After all bits have been transmitted, the bit counter reaches zero and the transmitter returns to the idle state. The TX line remains at logic HIGH until a new transmission request is receive.

8. UART RECEIVER DESIGN

8.1 Features

- Detection of UART Start Bit for frame synchronization.
- Reception of 8-bit serial data frames.
- Serial-to-parallel data conversion.
- Baud-rate-controlled data sampling.
- Half-bit synchronization for accurate bit detection.
- Stop Bit verification for frame validation.
- Storage of received data in the Receive Data Register.
- Interrupt generation upon successful data reception.

8.2 Working

The UART receiver continuously monitors the UART RX line while it remains in the idle state. When the RX line transitions from logic HIGH to logic LOW, the receiver detects the Start Bit and initiates the reception process.

To ensure accurate sampling, the receiver waits for half of a baud period and aligns itself with the center of the incoming data bits. It then samples one bit during each baud interval and shifts the received bits into the receive register.

After receiving all eight data bits, the receiver waits for the Stop Bit. If a valid Stop Bit (logic HIGH) is detected, the received byte is transferred to the Receive Data Register. The UART interrupt signal is then asserted to indicate that new data is available.

The interrupt remains active until it is acknowledged by the processor or the received data register is read. After a successful reception, the receiver returns to the idle state and waits for the next UART frame.

9. TESTBENCH DESIGN

9.1 FEATURES OF TESTBENCH

- Automatic generation of a 50 MHz system clock.
- Reset generation for UART initialization.
- Verification of both UART transmission and reception operations.
- Implementation of CPU read and write tasks for register access.

9.2 TESTCASES

TEST NO.	TX DATA	PURPOSE
1	0xA5	Alternating bit pattern
2	0x3C	Mid-range value
3	0xFF	All ones
4	0x00	All zeros

10. SIMULATION RESULTS

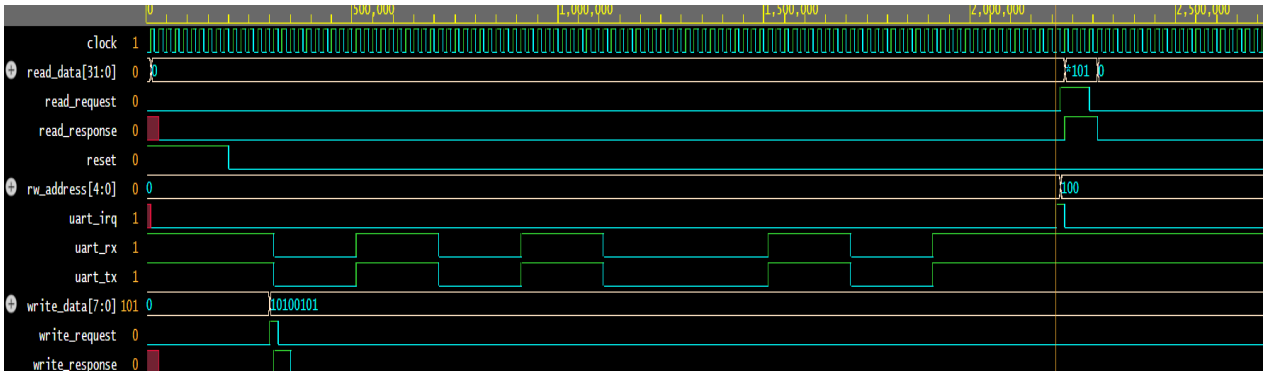

Figure 6 SIMULATION OUTPUT

11. CHALLENGES FACED

- Accurate baud-rate timing generation and synchronization.
- Proper start-bit detection during data reception.
- Correct serial-to-parallel and parallel-to-serial data conversion.
- Verification of UART transmission and reception using testbench stimuli.
- Debugging waveform timing issues during simulation.

12. ADVANTAGES

- Simple and cost-effective serial communication protocol.
- Requires only two communication lines (TX and RX).
- Supports full-duplex data transmission.
- Easy to implement using Verilog HDL.
- Widely used in microcontrollers, processors, and embedded systems.

13. DISADVANTAGES

- Lower communication speed compared to SPI.
- No built-in clock synchronization mechanism.
- Limited communication distance at high baud rates.
- Supports communication between a limited number of devices.
- Error detection capability is limited without additional mechanisms.

14. APPLICATIONS

- Serial communication between microcontrollers and embedded systems.
- Data exchange between processors and peripheral devices.
- Debugging and monitoring through serial terminals.
- Communication interfaces in FPGA-based designs.
- Industrial automation and IoT applications.

15. LEARNING OUTCOMES

- Gained practical experience in UART protocol implementation.
- Developed Verilog HDL coding and debugging skills.
- Learned UART frame generation and reception techniques.
- Understood baud-rate generation and timing control concepts.
- Acquired experience in testbench development and functional verification.

16. FUTURE ENHANCEMENT

- Addition of parity-bit generation and checking.
- Support for configurable baud rates.
- Implementation of transmit and receive FIFOs.
- FPGA-based hardware implementation and validation.
- Integration with processors and embedded systems for real-time communication.

17. CONCLUSION

The UART protocol was successfully implemented using Verilog HDL and functionally verified using EDA Playground. The design supports reliable asynchronous serial communication through UART transmission and reception mechanisms. A comprehensive testbench was developed to validate data transfer, register operations, and interrupt generation. Simulation results confirmed the correct operation of the UART module under various test conditions. The project provided valuable experience in digital design, protocol implementation, and hardware verification using Verilog HDL.

18. REFERENCES

[1] Universal Asynchronous Receiver Transmitter (UART) Protocol Specification and Technical Documentation.

[2] UART Communication Datasheet / Reference Manual used for protocol study and frame format analysis.

[3] Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd Edition, Pearson Education, 2003.

[4] M. Morris Mano and Michael D. Ciletti, *Digital Design*, 6th Edition, Pearson Education, 2017